Small challenges. Thirty thoughtful levels.

React Native screens. On-device Rust rules.
A calm space to notice, respond and pause.

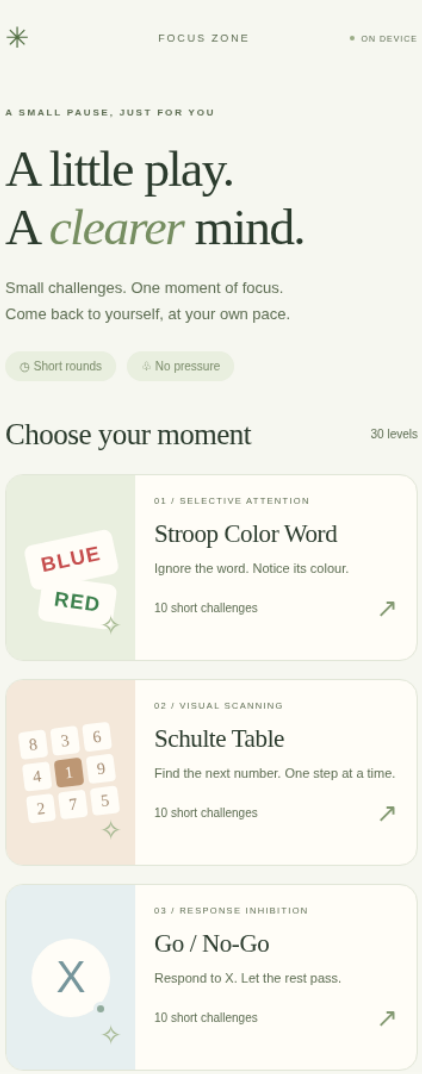
3 games · 10 levels each

Stroop Color Word
Selective attention

Schulte Table
Visual scanning

Go / No-Go
Response inhibition

Actual mobile source, native bridge adapters, reproducible rules and tests. A playable preview uses the same React Native UI and Rust engine.



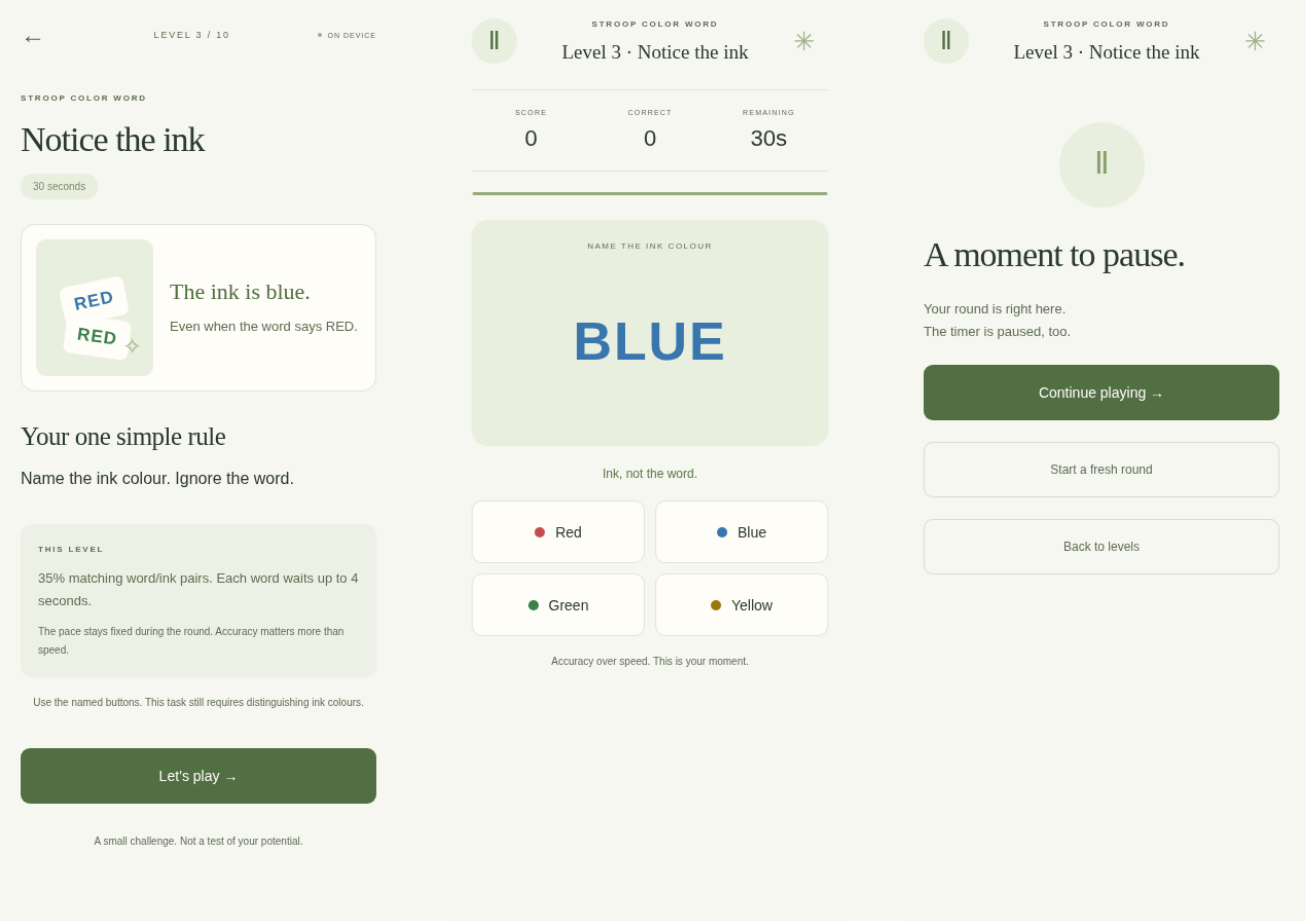
Readiness: production-oriented integration candidate. Shared UI/rules and native C-ABI checks pass; Kotlin/Swift app builds, host linkage and physical-device QA remain open. Build-tool security advisories also require host review.

DESIGN + LEVEL RULES + NATIVE INTEGRATION / VERSION 1

02 / MOBILE INTERACTION

One clear moment.

Quick to understand, touch-first and optional. Every round can be paused, replayed or left without pressure.



Ready. The exact level rule and duration are explained before starting.

Active. Prominent stimuli, large named controls and neutral feedback.

Paused. Active time freezes. The user decides when to resume.

A consistent session flow

Choose game → choose level → instructions → active → optional pause → completion → replay / next level / exit. Go / No-Go begins with a short, unscored three-letter warm-up on first play.

No daily streaks, locked levels, penalties tied to guilt, global rankings or clinical labels. Motion is subtle: a pop, ±3px shake and 12 particles; reduced motion disables it. Sound/haptics are host opt-ins, off by default.

Accessibility intent: safe areas, scrollable screens, named controls and at least 44px advanced-grid targets at 320px in emulation. VoiceOver/TalkBack, large text and physical-device accessibility still require native testing.

03 / GAME 01

Stroop Color Word

Ignore the word. Identify the ink. Four labelled responses: red, blue, green and yellow. Difficulty increases through interference and response windows, never changes mid-round.

LEVEL	TITLE	ROUND	WINDOW	MATCHING
1	First colours	30s	5s	75%
2	A gentle contrast	30s	4.5s	50%
3	Notice the ink	30s	4s	35%
4	Find your rhythm	45s	3.5s	25%
5	Let words pass	45s	3s	15%
6	A steadier focus	45s	2.8s	10%
7	Keep it clear	60s	2.4s	25%
8	Quiet the noise	60s	2s	20%
9	Stay with the colour	60s	1.7s	10%
10	The colour flow	60s	1.5s	10%

Resolution and feedback

Word and ink are randomized; matching and conflicting pairs both occur. Each answer or timeout reveals brief feedback, then advances after 240ms. The correct ink is never disclosed in instructions during the active trial.

Score: round(correct × 100 × answered accuracy). Accuracy is correct / answered. Missed trials are separate, not silently included as incorrect. Valid-response averages include correct and incorrect answers; timeouts are excluded.

This task requires distinguishing ink colours. Named buttons do not fully solve colour-vision differences; offer Schulte or Go / No-Go as alternatives. Global catalogue IDs: 1–10.

04 / GAME 02

Schulte Table

Find the next number shown on a fixed, randomized board. One complete sequence is one round; elapsed time is visible but there is no countdown or failure deadline.

LEVEL	TITLE	GRID	PATH
1	One small step	3 × 3	Ascending
2	Back to the beginning	3 × 3	Descending
3	Odd, then even	3 × 3	Odds, then evens
4	A wider view	4 × 4	Ascending
5	Reverse the path	4 × 4	Descending
6	A new number rhythm	4 × 4	Odds, then evens
7	From the outside in	4 × 4	Outside-in
8	A bigger picture	5 × 5	Ascending
9	A quiet reversal	5 × 5	Descending
10	The whole picture	5 × 5	Outside-in

Paths are explicit, not hidden tests

Ascending: 1, 2, 3… Descending: N, N−1… Odds first: 1, 3, 5… then 2, 4, 6… Outside-in: 1, N, 2, N−1… The next target remains visible. Wrong taps never advance or reshuffle the board; found cells become check marks.

Score: round(10,000 × correct / attempted). Save only after the entire sequence is completed. Compare times within the same level only. The three 5×5 levels can be hidden with allowAdvancedGrid=false.

Larger grids and new paths are deliberate extensions beyond the original ascending 3×3/4×4 MVP. Validate their usability before enabling them in a production rollout. Global IDs: 11–20.

05 / GAME 03

Go / No-Go

Respond to X. Let A, B, K and O pass. Shorter displays and higher target frequency practise noticing before responding. The rule and pace stay fixed during a round.

LEVEL	TITLE	ROUND	LETTER	GAP	TARGETS
1	Notice and pause	30s	1.5s	0.5s	50%
2	Let it pass	30s	1.4s	0.45s	50%
3	Choose your response	30s	1.3s	0.4s	60%
4	A mindful beat	45s	1.2s	0.4s	60%
5	Room for a pause	45s	1.1s	0.35s	60%
6	A steady hand	45s	1s	0.35s	70%
7	Stay with the signal	60s	0.9s	0.35s	70%
8	A quicker rhythm	60s	0.8s	0.3s	70%
9	Notice before you act	60s	0.75s	0.3s	75%
10	The mindful flow	60s	0.65s	0.3s	75%

Responding and withholding stay separate

Target + response = hit. Target + no response = miss. Non-target + response = false alarm. Non-target + no response = correct withholding. Each stimulus accepts at most one response. Randomized 20-trial bags match the configured target ratio over a full bag; partial rounds may differ.

Results: hit rate and false-alarm rate are shown independently. Composite score = round((hits + correct withholds) × 100 × overall trial accuracy). Mean response uses successful hits only. The X/A/X warm-up is never stored or ranked.

An unfinished final trial is explicitly finalized at the end of the session. Global IDs: 21–30.

06 / ENGINEERING

Native by design.

No WebView and no Rust HTTP service: React Native renders the game; Rust runs locally in the mobile app.

Host app — navigation, account scope, theme, safe areas, secure storage

React Native FocusZone — touch UI, accessible controls, native-driven feedback

FocusClient — ordered commands, typed errors, protocol/schema validation

TurboModule — dispatch(JSON) → Promise<JSON>

Android: Kotlin serial executor → UniFFI/JNA
iOS: ObjC++ serial queue → Swift → UniFFI/C ABI

Rust FocusEngine — mutex-owned session, monotonic timing, rules, RNG, scoring

One catalogue. One source of truth.

levels.json is imported by TypeScript and compiled into Rust. Both artifacts must be rebuilt together; catalogue mismatches are rejected. Session and trial IDs prevent stale/duplicate inputs. New starts discard unfinished rounds; completed result IDs deduplicate save retries.

Pause, backgrounding and performance

Rust Instant is authoritative on device. Pause stops active duration; returning to the app does not auto-resume. Host route blur, RN AppState and native Android/iOS lifecycle hooks interrupt rounds. Returning never auto-resumes. Snapshot polling is single-flight at up to 12.5Hz; visual animation is independent and native-driven.

Timing caveat: UI presentation may lag a native stimulus by roughly one 80ms poll interval plus bridge/render latency. Response averages start at native stimulus generation. They are practice feedback, not calibrated display-onset or clinical measurements.

07 / VALIDATION

Tested source. Explicit gates.

Verified in this environment, with the boundary between code validation and a shipping native app kept clear.

CHECK	STATUS	WHAT IT MEANS
33 Rust tests	Passed	Rules, scoring, sequences, timing and input protections
12 TypeScript tests	Passed	Ordering, typed errors, scoped storage and retry
Typecheck + Clippy	Passed	Strict TS; Rust warnings treated as errors
Rust iOS / Android checks	Passed	Cross-target type checking, not full native builds
Native C-ABI calls	Passed	Linux UniFFI; real engine, concurrent polling
RN spec generation	Passed	Android Java and iOS ObjC++ interface contracts
Shared UI / all 30 levels	Passed	RN Web + Rust WASM, touch emulation
Native app integration	Open	Kotlin/Swift compilation, autolinking, release linkage
Physical accessibility / FPS	Open	Real device, screen reader, font and latency testing

Frame scheduling, not a device guarantee

Active RN Web + Rust WASM games measured about 59.7fps over 180 intervals on desktop/mobile emulation, with p95 frame intervals around 16.7–16.8ms. Native device performance is not certified by those numbers.

Security review remains open: npm audit reports high-severity affected package entries in the Metro/micromatch/braces tooling graph. No safe automatic fix was available; the suggested force fix downgraded RN and was not applied. Review/mitigate in host CI; do not expose Metro or feed untrusted patterns. Rust advisory and license scans are still required.

08 / INTEGRATION

Ready for your host project.

The source package includes native adapters, generated bindings, build scripts, test evidence and detailed integration/release documents. No existing SWA project was supplied or modified.

Android

Install the local/workspace package. Set ANDROID_NDK_HOME, install cargo-ndk, then run scripts/build-android.sh. The script builds ARM64 and x86_64 native libraries with 16KB alignment for Rust. Rebuild the real RN host, verify codegen, R8 and all dependency ABIs/page sizes.

iOS

On macOS/Xcode run scripts/build-ios.sh to generate Swift bindings and a device/simulator XCFramework. Install the local CocoaPods target and rebuild the host. Verify module-map visibility, Swift/ObjC++ headers, ReactCodegen paths, autolinking and release dead stripping.

Mount through the app’s existing services

Use one FocusClient per native engine. Inject a repository keyed to the real account/profile, reuse the app’s SafeAreaProvider and call onExit through existing navigation. Remount on account change. Reuse secure/SQLite/AsyncStorage services according to the host’s established architecture; no new database is imposed.

Privacy: up to 300 completed, non-practice results per scope. IDs, level/version, wall-clock start/end metadata, active duration and task metrics only; no raw stimulus sequences. Writes are ordered and idempotent; corruption is reported instead of overwritten. A storage namespace is not authentication or encryption.

Before shipping

Finish native builds, physical-device timing and accessibility QA, dependency/security review, account-switch/privacy tests, advanced-grid usability and the supplied release checklist. Keep Focus optional and separate from morning/evening reflection.

These tasks do not measure self-awareness, diagnose mental-health conditions or prove improvements in everyday concentration. The appropriate promise is a short, meaningful practice moment.